

CS 650 – Full Stack Application Develop

9-1 Project Two: Operationalizing and Validating Application Readiness

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

September 1, 2026

Introduction

RecipeRack is an existing full-stack web application that combines recipe management with an ingredient-based discovery feature. The purpose of Project Two was not to rebuild the application, but to operationalize it by adding a repeatable quality process around the software. The completed work focused on automated functional and nonfunctional testing, continuous integration and continuous delivery practices, observability and error handling, and formal release management.

The finished project demonstrates that RecipeRack can be verified at multiple levels before release. Backend unit tests validate core service logic, Playwright end-to-end tests exercise complete user workflows through the user interface, API, and database, GitHub Actions automatically performs build-and-test checks on repository changes, runtime logging provides visibility into requests and failures, and the validated code has been packaged as the versioned RecipeRack v1.0.0 release.

Project Requirements and Traceability

The project requirements define two primary user stories. US-1 requires users to discover recipes based on ingredients they have available, including autocomplete, multiple ingredient selection, match classification, and ordering of full, partial, and no-match results. US-2 requires users to manage the recipe collection by creating, editing, searching for, and deleting recipes. The project also defines two nonfunctional requirements: NFR-1 for API and recipe-matching performance, and NFR-2 for input validation and HTTP error handling.

Requirement	Purpose	Automated Evidence	Result
US-1 / F1	Ingredient-based recipe discovery and match classification	Playwright ingredient discovery tests	Passed
US-2 / F2	Create, edit, search/view, and delete recipe lifecycle	Playwright recipe lifecycle test	Passed
NFR-1 / NF1	Recipe matching under load, including 500-recipe dataset	Playwright performance test	Passed
NFR-2 / NF2	Reject malformed input and missing resources correctly	Playwright validation/API tests	Passed
Service layer	Recipe CRUD, ingredient normalization, search, and matching logic	Backend automated unit tests	24 passed
CI	Automatically verify repository changes	GitHub Actions RecipeRack CI workflow	Success
Release	Produce a traceable versioned release	GitHub Release v1.0.0	Published

Automated Backend Testing

The backend test suite verifies the application logic independently of the browser. This layer is important because it isolates defects in recipe and ingredient behavior before those defects appear in complete user workflows. The executed suite contained 24 tests across eight suites, and all 24 tests passed with zero failures, cancellations, skips, or unfinished tests.

The backend coverage includes recipe creation, retrieval, listing, searching, updating, and deletion. It also checks ingredient behavior, including normalization to lowercase and trimmed values, reuse of existing ingredient records instead of unnecessary duplication, prefix-based ingredient filtering, and empty results when no ingredient matches. The matching logic verifies all three required classifications: full, partial, and none. It also verifies that full matches are ordered before partial matches and partial matches before no matches.

The backend suite additionally exercises a 500-recipe dataset as a performance-oriented proxy for NFR-1. This is valuable because performance is treated as a testable requirement rather than an assumption. Passing these tests establishes a stable service-layer foundation for the end-to-end tests.

End-to-End Functional Testing

Playwright was used to execute the required end-to-end scenarios. These tests are especially important for this project because the requirements specify that testing should exercise the application from the interface through the API to the database and back. The final Playwright run executed nine tests using four workers, and all nine tests passed.

F1: Recipe Discovery Flow

The F1 coverage validates the ingredient-discovery workflow associated with US-1. The test verifies that the autocomplete control suggests matching ingredients, and that selecting an ingredient creates the expected selection state. It also verifies that recipe matches classifications update as ingredients are added and removed. This directly supports the core purpose of the What Can I Make? Feature: helping a user understand which recipes are possible based on currently available ingredients.

The underlying service tests complement the browser workflow by confirming the classification rules themselves. A recipe is classified as a full match when the selected ingredient set satisfies all recipe ingredients, as a partial match when only some required ingredients are present, and as no match when none are present. The ordering logic is also tested, so the most useful results appear first.

F2: Recipe Lifecycle

The F2 test validates the complete CRUD lifecycle required by US-2. The automated browser scenario creates a recipe with multi-step instructions, edits the recipe title, confirms that the updated recipe is visible, opens the recipe details, and deletes the recipe. This is a stronger verification than testing isolated buttons because the test follows a realistic user journey and confirms that changes persist across the application layers.

During development, this scenario initially exposed a database foreign-key constraint problem while updating a recipe. The failing test could not find the updated recipe row after the save action because the backend update operation failed. The service-layer update behavior was corrected, and the full test suite was rerun. The final result was nine of nine Playwright tests passing. This demonstrates the practical value of automated regression testing: the test did not

merely document expected behavior; it detected an integration defect that was then fixed and verified.

Nonfunctional Testing

NF1: Performance

NFR-1 requires API responses within 500 milliseconds under normal conditions and requires recipe matching to complete within one second when evaluating up to 500 recipes. The NF1 automated scenario loads a realistic 500-recipe dataset and evaluates the `/api/discover` behavior. The final Playwright run reports the NF1 scenario as passed. This provides repeatable evidence that the matching path satisfies the project's defined performance test under the supplied test conditions. Performance results should still be interpreted in context. Local and CI test environments are controlled environments and do not reproduce every production factor such as network latency, concurrent traffic, or hosting resource contention. For that reason, the automated performance test serves as a release gate and baseline, while runtime monitoring should continue to observe latency after deployment.

NF2: Validation and Error Handling

NFR-2 requires validation on both client and server and appropriate HTTP responses for invalid requests and missing resources. The automated NF2 scenarios verify that `POST /api/recipes` returns HTTP 400 when the title is missing, ingredients are missing, instructions are missing, or the request body is absent. The suite also verifies that a GET request for a nonexistent recipe identifier returns HTTP 404. A separate manual API check was also performed against `/api/recipes/999999`. The backend returned HTTP/1.1 404 Not Found with a JSON error response of `{"error":"Not found"}`. The response also included security-related HTTP headers and rate-

limit information. Together, the automated and manual evidence shows that the application handles invalid and missing-resource conditions deliberately rather than failing silently or returning misleading success responses.

Observability and Runtime Monitoring

Operational quality requires visibility after the application starts, not only before it is released. RecipeRack's backend provides request-level runtime logging that records the HTTP method, requested route, response status, and request duration. During verification, the server logged requests such as GET /api/recipes/999999 -> 404 (1 ms). This type of logging supports troubleshooting because it immediately shows which route was requested, whether the request succeeded, and how long the server took to respond.

The HTTP response evidence also shows several defensive controls, including Content-Security-Policy, Cross-Origin-Opener-Policy, Cross-Origin-Resource-Policy, Referrer-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, and X-XSS-Protection headers, as well as rate-limit headers. These controls do not replace application-level validation, but they strengthen the operational posture of the HTTP service.

For continued operation, the most useful monitoring signals are request volume, HTTP 4xx and 5xx rates, endpoint latency, recipe-discovery response time, CI failure frequency, and release health after deployment. A rise in 500 responses or latency beyond the NFR-1 threshold should trigger investigation. Repeated 400 responses may also identify usability or client-validation problems even when the server is technically functioning as designed.

Continuous Integration and Delivery

The repository includes a GitHub Actions workflow named RecipeRack CI. The workflow is stored `.github/workflows/ci.yml` so the automation configuration is version controlled with the application. When code is pushed, GitHub Actions creates a clean runner environment and executes the defined build-and-test process. This reduces dependence on a developer remembering to run every check manually and makes the verification process repeatable.

The final workflow run was triggered by a push to the main branch at commit `4677c1f`. GitHub reported Status: Success, with the Build and Test RecipeRack job completing successfully. The overall workflow was completed in 51 seconds. A green workflow is significant because it connects the repository state to objective automated evidence: the committed version passed the configured quality gate in a fresh CI environment.

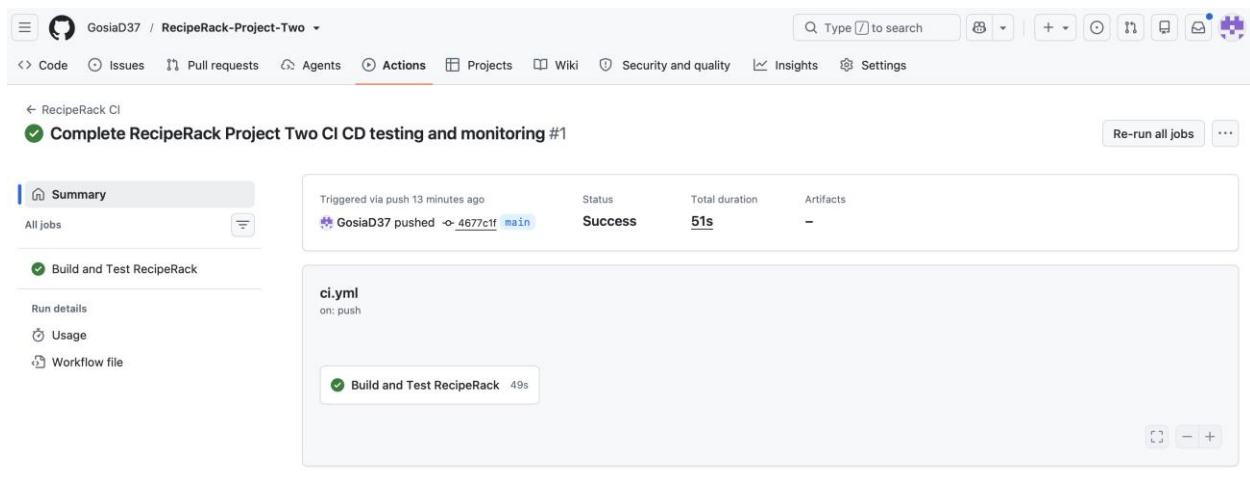


Figure 1. GitHub Actions workflow summary showing a successful RecipeRack CI run for commit `4677c1f`.

Release Management

After the automated checks passed, the validated code was released as RecipeRack v1.0.0. The release uses the semantic-style version tag `v1.0.0` and points to commit `4677c1f`, the same

commit represented in the successful CI evidence. This creates traceability between source code, automated verification, and published release.

The release page identifies v1.0.0 as the latest release and provides automatically generated source-code archives in ZIP and TAR.GZ formats. Creating a formal release is preferable to treating an arbitrary branch state as a deployment candidate because the version tag establishes a stable reference that can be reviewed, downloaded, reproduced, or compared with future versions.

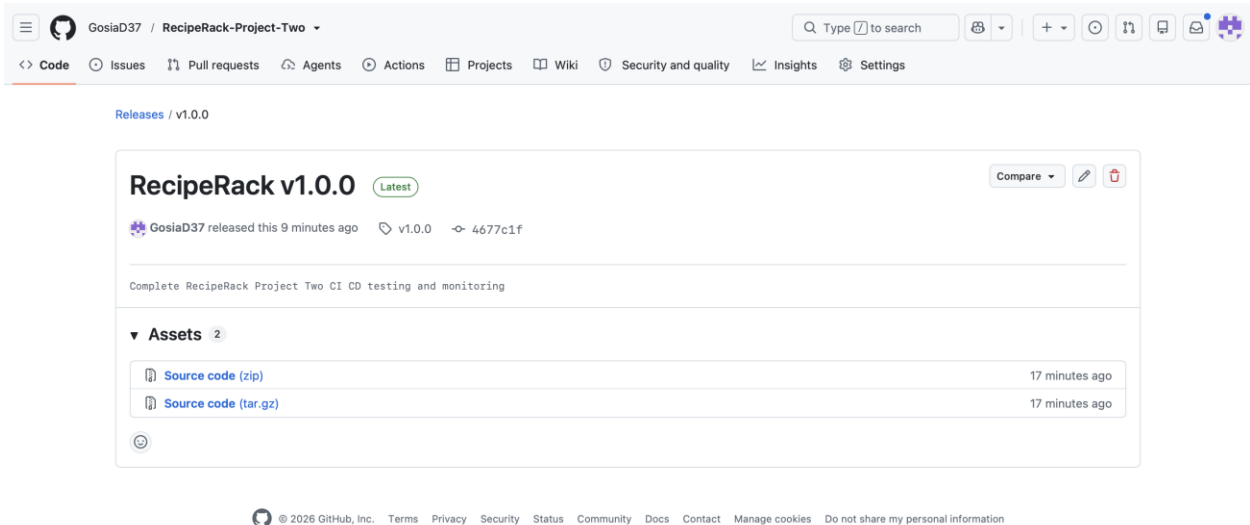


Figure 2. Published GitHub release for RecipeRack v1.0.0, linked to commit 4677c1f.

CI/CD and Release Quality Controls

The completed workflow forms a basic release pipeline: source changes are committed to Git, pushed to GitHub, automated checks execute through GitHub Actions, and a versioned release is

created only after the verified code is ready. The relationship between these stages improves reliability because failures can be detected before a version is presented as a release.

The production deployment process should preserve the same principle. A release candidate should be promoted only after required tests to pass; the build is reproducible, monitoring is ready, and rollback is possible. If a future release causes elevated server errors, unacceptable latency, or failed core recipe workflows, the team should stop promotion and restore the previous known-good version while the defect is investigated.

Evidence of Final Project Completion

The final project evidence demonstrates completion across the major operational areas. The backend suite passed 24 tests, confirming core service and matching behavior. The Playwright suite passed all nine end-to-end and nonfunctional tests, including recipe lifecycle, ingredient discovery, validation, missing-resource handling, and the 500-recipe performance scenario. The backend successfully produced observable request logs and appropriate 404 behavior. The repository was placed under Git version control and pushed to GitHub. GitHub Actions completed the RecipeRack CI workflow successfully. Finally, the validated commit was published as RecipeRack v1.0.0.

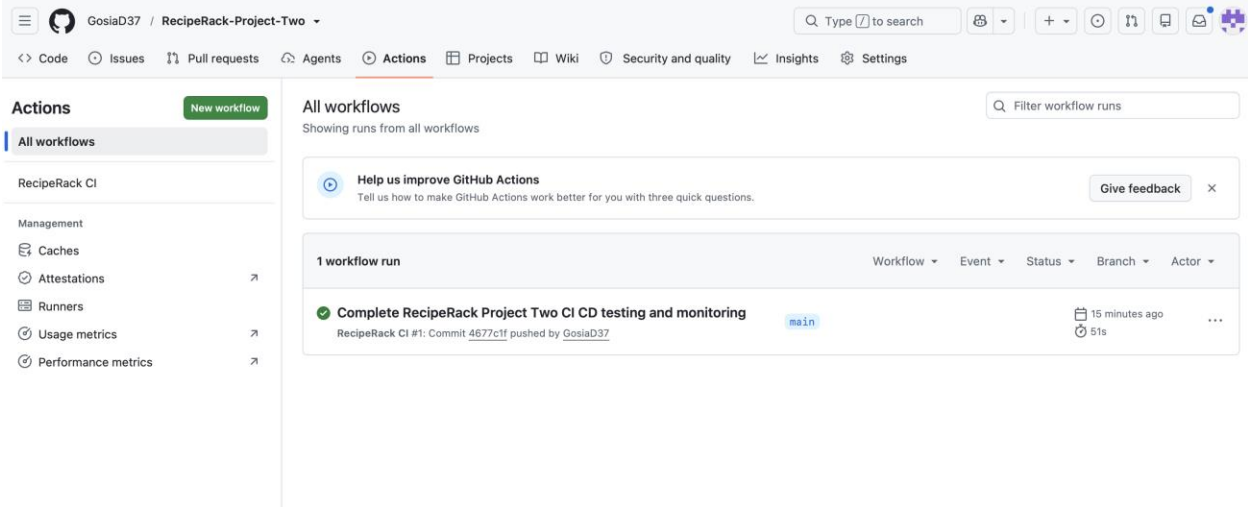


Figure 3. GitHub Actions workflow history showing the successful RecipeRack CI run on the main branch.

Risk Assessment and Future Improvements

The current automated suite provides strong coverage of the required user stories and nonfunctional requirements, but a passing pipeline does not eliminate every software risk. Automated tests only verify behaviors represented in the test suite. Browser differences, accessibility issues, unexpected user behavior, production configuration, network failures, and infrastructure-specific performance can still introduce problems.

Several improvements would strengthen the operational process further. Branch protection could require the CI check to pass before changes are merged into main and could require at least one peer review. Additional tests could cover multiple browsers, accessibility, concurrent requests, database recovery, and deliberate server-error scenarios. Production monitoring could collect structured metrics for latency and error rates and establish alert thresholds. The release workflow could also automatically create a build artifact and release notes after a version tag is approved. These improvements would extend the same quality principles demonstrated in Project Two as RecipeRack moves from a course project toward a more production-like lifecycle.

Conclusion

Project Two transformed RecipeRack from an existing full-stack application into a more operationally mature software product. The project connects requirements to automated verification, verifies both functional behavior and nonfunctional quality, provides runtime visibility, uses continuous integration to validate repository changes, and creates a traceable versioned release. The final evidence is consistent across these stages: 24 backend tests passed, nine Playwright tests passed, the GitHub Actions workflow completed successfully, and RecipeRack v1.0.0 was published from the validated commit. The most important outcome is not simply that the application works at one moment in time. The project establishes a repeatable process for determining whether future changes are safe to integrate and release. Automated tests provide fast feedback; CI makes those checks consistent, observability supports diagnosis during operation, and versioned releases provide traceability and recovery points. Together, these practices improve software quality, reduce release risk, and create a stronger foundation for continued development.